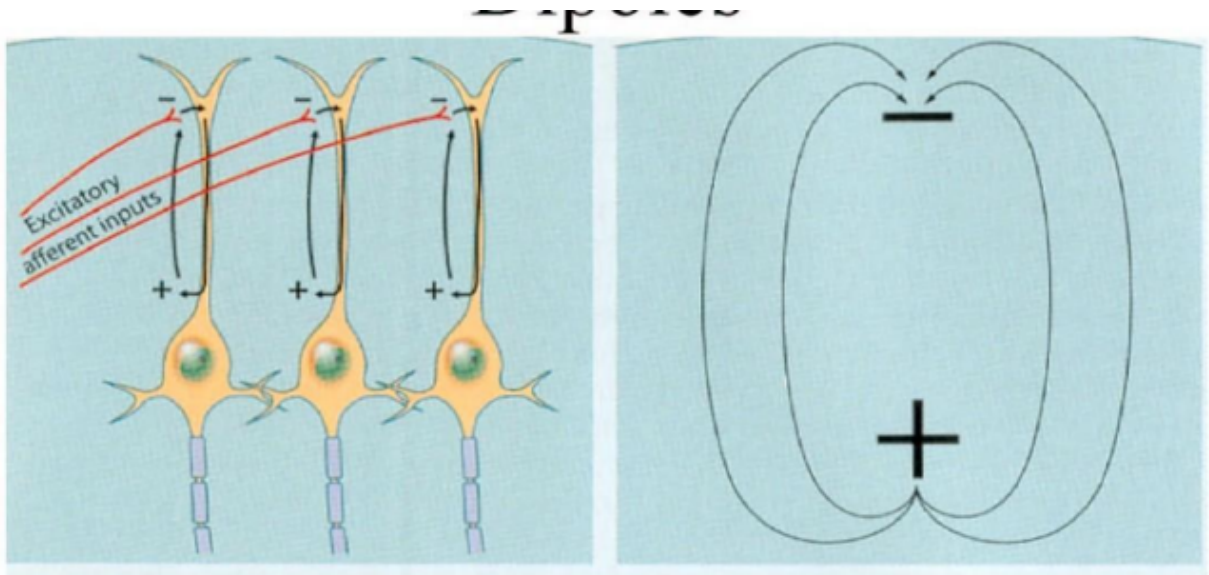


EEG is the process of creating an Electrical Brain Picture. Historically, the field progressed from Richard Caton's 1875 observations of animal brains to Hans Berger's 1924 discovery of human brain waves using radio equipment. The primary advantage of EEG is its superior temporal resolution, allowing clinicians to monitor alertness, investigate sleep disorders, and locate seizure origins in epilepsy patients.



Neuroimaging and Neural Physics

EEG is unique among neuroimaging modalities (like fMRI, PET, and MEG) because it does not require the participant's head or body to be fixed. This portability enables the monitoring of brain functions in unconstrained, real-world environments like homes or workplaces.

The electrical signals detected on the scalp originate from the postsynaptic potentials of pyramidal neurons in the cerebral cortex. For these signals to be strong enough for scalp detection, two conditions must be met: the neurons must be physically aligned, and their activity must be synchronous.

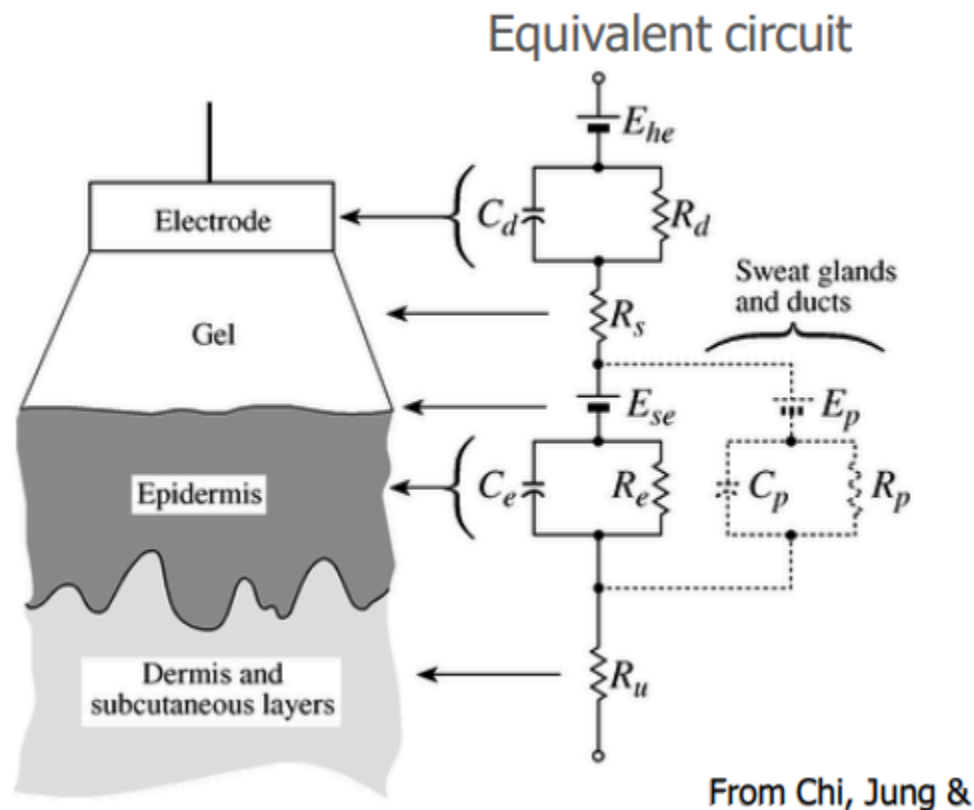
Acquisition Basics

A single scalp electrode detects the neural synchronies of millions of underlying neurons. The acquisition process typically involves electrode caps, conductive jelly (to reduce impedance), and an amplifier unit connected to a computer for data recording.

The Skin-Electrode Interface

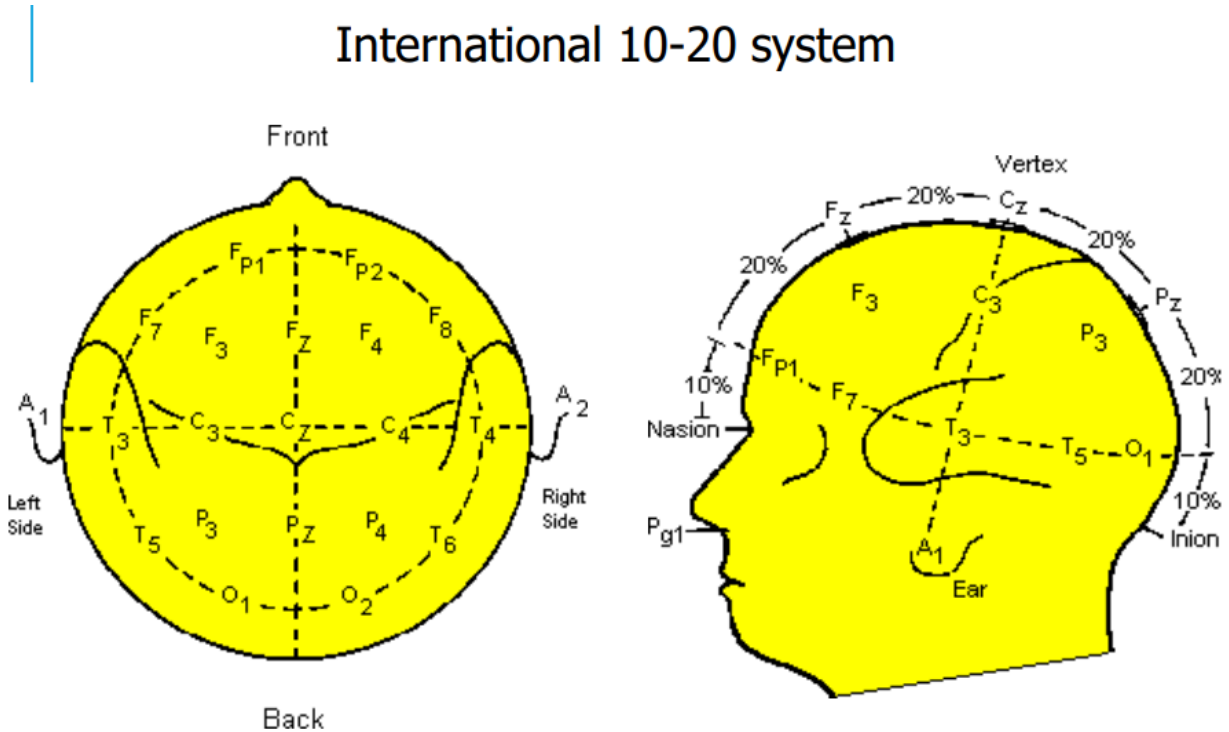
The coupling between the skin and the electrode is modeled as a complex layered RC (Resistor-Capacitor) structure.

- Circuit Equivalent of Body Substances:** Different biological layers are modeled based on their electrical properties. The Metal-Electrode interface creates a half-cell potential (V_{hc}), while the Electrolyte Gel and Epidermis are represented as parallel RC elements ($R_d||C_d$ and $R_e||C_e$) because they possess both resistive and capacitive (insulating) properties.
- Body Resistance vs. Capacitance:** In this model, the Body (dermis and deeper subcutaneous layers) is represented solely as a resistance (R_{body} or R_u) without parallel capacitance. This is because internal tissues consist of highly conductive ionic fluids and lack the thin, insulating dielectric barriers (like the dead cells of the stratum corneum in the epidermis) required to create significant capacitance at EEG frequencies.
- Chi, Jung & Cauwenberghs (2010):** This pivotal paper provides a methodological review of dry and non-contact biopotential electrodes. It establishes that optimal low-noise signal reception is a dichotomy, it can be achieved either through infinite coupling conductance (standard low-resistance wet sensing) or infinite coupling impedance (capacitive non-contact sensing). Practically, this means researchers must either abrade the skin to reach a low resistance (5-10k Ω) or use amplifiers with extremely high input impedance.



Electrodes and Montage

While standard wet electrodes provide high signal quality, they are messy and degrade over time. Alternatives include rigid metal, dry foam, and non-contact sensors, though many are highly sensitive to motion artifacts. Most modern wearable devices (e.g., Muse, OpenBCI, g.tec) utilize these varied sensor types. Electrode placement is standardized globally via the International 10-20 system, which uses percentages of the head's dimensions to ensure consistent mapping.



EEG Experiment Setup and Process

A standard EEG experiment involves a participant seated in an electromagnetically shielded recording chamber to block external interference. The participant wears an electrode cap while their head is stabilized by a fixed chin rest to minimize motion artifacts. During the session, visual or auditory stimuli are presented, and the subject often interacts with a keypress response pad. Precision tools are used to measure the exact orientation and position of the electrodes to ensure data accuracy.

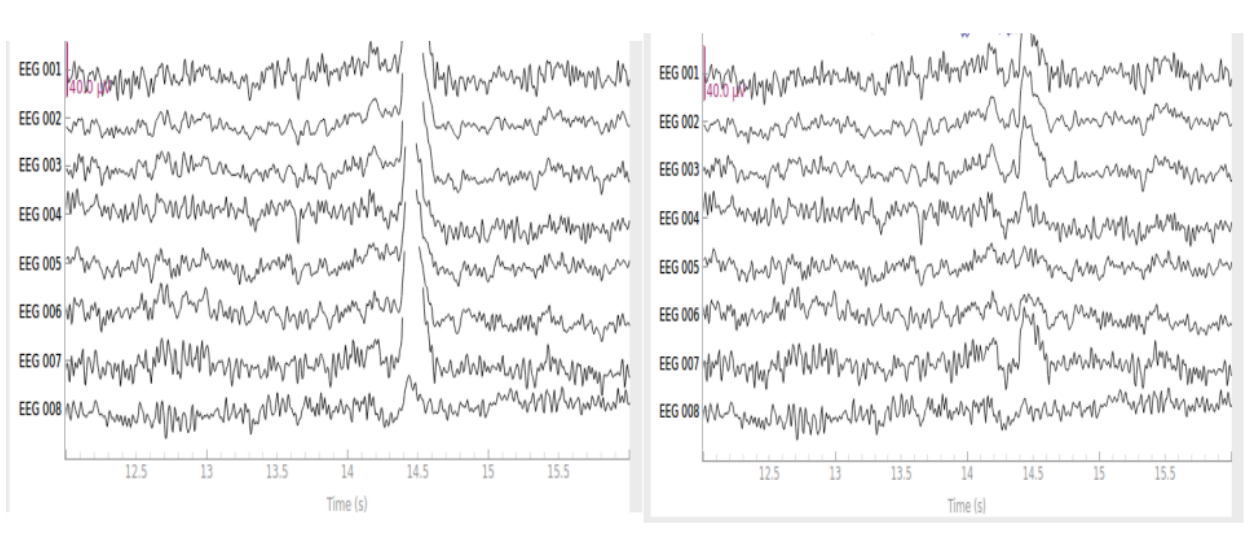
Capabilities and Challenges

EEG excels at providing the precise timing of neural activity but struggles with precise location due to the inverse problem—determining the exact internal source of a signal measured on the surface. Major challenges include pervasive artifacts from eye movements and muscle activity.

Preprocessing and Noise Removal

Preprocessing transforms messy raw data into a clean signal by removing outliers and noise.

- **Grid Frequency (50Hz):** Electrical power line noise is removed using a 50Hz notch filter, which attenuates that specific frequency without affecting the surrounding brain activity.
- **Eye Blinks (EOG):** These create large, low-frequency voltage shifts that are identified and removed during preprocessing to prevent them from distorting the brain wave data.
- **Jaw Movement (EMG):** Muscle activity creates high-frequency noise that is typically attenuated using low-pass filters during the noise filtering stage.

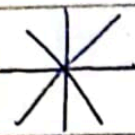


Performance Parameters

The presentation identifies 12 parameters used to evaluate signal quality and algorithm performance:

1. **SNR (Signal to Noise Ratio):** Measures signal clarity against background noise.
2. **RMSE (Root Mean Square Error):** Quantifies the average error magnitude.
3. **CC (Correlation Coefficients):** Normalized (CCN) and Weighted (CCW) values measure how closely the processed signal matches a reference.
4. **AE (Absolute Error):** Includes Mean (AEM) and Maximum (AEMAX) values.
5. **RD (Root Mean Square Difference):** Expressed as Normalized (RDN) or Percentage (RDP).
6. **MI (Mutual Information):** A statistical measure of how much information is shared between signals.
7. **n (Percentage Reduction):** Measures the decrease in the coefficient of correlation.
8. **MAXN:** The normalized maximum absolute error.

Date: / /



ECT-107

Lab Report Feb 08

$$SNR = 10 \log_{10} \left[\frac{\sum_{l=1}^L (v(l) - \mu_v)^2}{\sum_{l=1}^L (v(l) - z(l))^2} \right]$$

↓
decibel scale

→ clean signal power (variance)
→ power of residual signal

$$AE_{max} = \max |v(l) - z(l)|$$

→ max absolute error

$$AE_M = \frac{\sum |v(l) - z(l)|}{L}$$

→ sum of all absolute errors
→ averaged out

$$MAXN = \frac{\max |v(l) - z(l)|}{\max(v(l)) - \min(v(l))}$$

→ max error
→ total range of $v(l)$

$$RMSE = \sqrt{\frac{\sum |v(l) - z(l)|^2}{L}}$$

→ square
→ root
→ mean

$$RDp = \frac{\sum |v(l) - z(l)|^2}{\sum (v(l) - 1024)^2}$$

1024 is ADC baseline

$$RDN = \frac{\sqrt{\sum (v(l) - z(l))^2 / L}}{\max(v(l)) - \min(v(l))}$$

→ RMS divided by signal range (normalized)

$$CCN = \frac{\frac{1}{L} \sum (v(l) - \mu_v) \sum (z(l) - \mu_z)}{\sqrt{\frac{1}{L} \sum (v(l) - \mu_v)^2 \frac{1}{L} \sum (z(l) - \mu_z)^2}}$$

→ Pearson Correlation Coeff. 1: perfect corr.
0: no linear rel.

$$\eta = \left[1 - \frac{1 - S_{vz}}{1 - S_{vx}} \right] \times 100\%$$

S_{vz} : corr. b/w clean & jittered
 S_{vx} : corr. b/w

$$MI = \sum_i \sum_j P_{(x,z)}(i,j) \log_2 \frac{P_{(x,z)}(i,j)}{P_x(i) P_z(j)}$$

Identification and Removal of Physiological Artifacts From Electroencephalogram Signals: A Review

The review paper (IEEE Access, 2018) highlights that while EEG offers high temporal resolution, its utility is often compromised by non-neuronal artifacts. These are undesirable electrical potentials that originate from sources other than the brain, distorting the signal and potentially leading to clinical misinterpretation.

Key Physiological Artifacts:

- **Ocular Artifacts (EOG):** Generated by eye blinks and movements. These are high-amplitude and primarily affect frontal electrodes (e.g., Fp1, Fp2).
- **Muscle Artifacts (EMG):** Caused by jaw clenching, swallowing, or neck movements. These present as high-frequency noise (>30 Hz) and are often the most difficult to isolate.
- **Cardiac Artifacts (ECG):** Occur when electrodes are placed near blood vessels, appearing as regular, rhythmic pulses that can be mistaken for periodic brain activity.

Common Removal Methodologies

Modern signal processing utilizes several standard techniques to clean raw EEG data:

- **Regression Methods:** Use reference channels (like EOG or ECG) to estimate and subtract the artifact from the EEG signal.
- **Independent Component Analysis (ICA):** Decomposes the signal into statistically independent components, allowing researchers to manually or automatically reject components identified as noise.
- **Filtering:**
 - **High-pass/Low-pass:** Removes slow drifts and high-frequency muscle noise.
 - **Notch Filter (50/60 Hz):** Specifically targets power line interference.
- **Wavelet Transform:** Effective for non-stationary signals where artifacts overlap in frequency with neural activity.

Analysis of Referenced Datasets

The provided links offer various ground truth scenarios for testing artifact removal algorithms:

- **Semi-Simulated EEG/EOG Dataset (Mendeley):** This is a unique dataset where clean, artifact-free EEG was manually contaminated with ocular artifacts using a realistic head model. It allows researchers to calculate the exact error (RMSE) of their removal algorithms because the pure signal is known.
- **MIT-BIH Polysomnographic Database (slpdb):** A clinical database containing multiple physiological signals (EEG, ECG, Respiration). It is primarily used to study cardiac artifacts in sleep EEG and apnea detection.

- **EEG During Mental Arithmetic (eegmat):** Contains data from subjects performing serial subtractions. This dataset is useful for observing how cognitive load affects neural rhythms and how muscle tension (from concentration) introduces artifacts.
- **Motion Artifact Contaminated fNIRS and EEG:** Specifically designed for mobile brain imaging. It includes data from a disturbed transducer (moved intentionally to create noise) and an undisturbed transducer acting as the ground truth.
- **Mobile Brain-Body Imaging (MoBI - Nature):** Features data from subjects walking on treadmills. This represents the frontier of EEG, focusing on removing the intense motion artifacts generated during physical locomotion to isolate motor-related brain signals.

Further attached is the notebook implementation of these concepts: This notebook demonstrates the practical application of biomedical signal processing techniques to real-world EEG data. Building upon the foundational methodologies provided in the initial classroom demonstration, this work adapts and applies those concepts to the PhysioNet Mental Arithmetic Task dataset.

The primary objective is to clean raw EEG signals by identifying and removing common noise sources, such as power line interference and ocular (eye-movement) artifacts, to ensure data reliability for further neurological analysis. Key Adaptations from the Original Notebook To transition from the simulated environment to real-world data, the following technical updates were implemented:

- **File Format Integration:** Switched from loading MATLAB (.mat) files to processing standardized European Data Format (.edf) files using the MNE-Python library.
- **Sampling Frequency Calibration:** Reconfigured all temporal and spectral calculations from the original 200 Hz to the dataset's native 500 Hz sampling rate.
- **Amplitude Normalization:** Adjusted the data scaling to microvolts (uV) to maintain consistency with clinical EEG standards and the original demonstration's visualization scales.
- **Dynamic Time-Windowing:** Scaled the analysis windows from 700 samples to 1,750 samples to ensure a consistent 3.5-second signal duration for comparison between baseline (pure) and task-related (contaminated) states.
- **Enhanced Anatomical Labeling:** Transitioned from generic channel numbering to the International 10-20 system labels (e.g., Fp1, Fz, Cz), allowing for precise anatomical tracking of signal artifacts.

```
from matplotlib import pyplot as plt
import pandas as pd
import numpy as np
from scipy import signal

!pip install mne
import mne
```

```
Collecting mne
  Downloading mne-1.11.0-py3-none-any.whl.metadata (15 kB)
Requirement already satisfied: decorator in /usr/local/lib/python3.12/dist-packages (from mne) (4.4.2)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from mne) (3.1.6)
Requirement already satisfied: lazy-loader>=0.3 in /usr/local/lib/python3.12/dist-packages (from mne) (0.4)
Requirement already satisfied: matplotlib>=3.8 in /usr/local/lib/python3.12/dist-packages (from mne) (3.10.0)
Requirement already satisfied: numpy<3,>=1.26 in /usr/local/lib/python3.12/dist-packages (from mne) (2.0.2)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from mne) (26.0)
Requirement already satisfied: pooch>=1.5 in /usr/local/lib/python3.12/dist-packages (from mne) (1.9.0)
Requirement already satisfied: scipy>=1.11 in /usr/local/lib/python3.12/dist-packages (from mne) (1.16.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from mne) (4.67.2)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.8->mne) (0)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.8->mne) (0)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.8->mne) (0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.8->mne) (0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.8->mne) (11.3)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.8->mne) (0)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.8->mne) (0)
Requirement already satisfied: platformdirs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from pooch>=1.5->mne) (0)
Requirement already satisfied: requests>=2.19.0 in /usr/local/lib/python3.12/dist-packages (from pooch>=1.5->mne) (2)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2->mne) (3.0.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib) (0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.19.0->pooch) (0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.19.0->pooch) (0)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.19.0->pooch) (0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.19.0->pooch) (0)
Downloading mne-1.11.0-py3-none-any.whl (7.5 MB)
7.5/7.5 MB 36.3 MB/s eta 0:00:00
Installing collected packages: mne
Successfully installed mne-1.11.0
```

```
from scipy.io import loadmat
from scipy.fft import fft
```

```
import mne

raw = mne.io.read_raw_edf('Subject00_1.edf', preload=True)
data = raw.get_data()
```

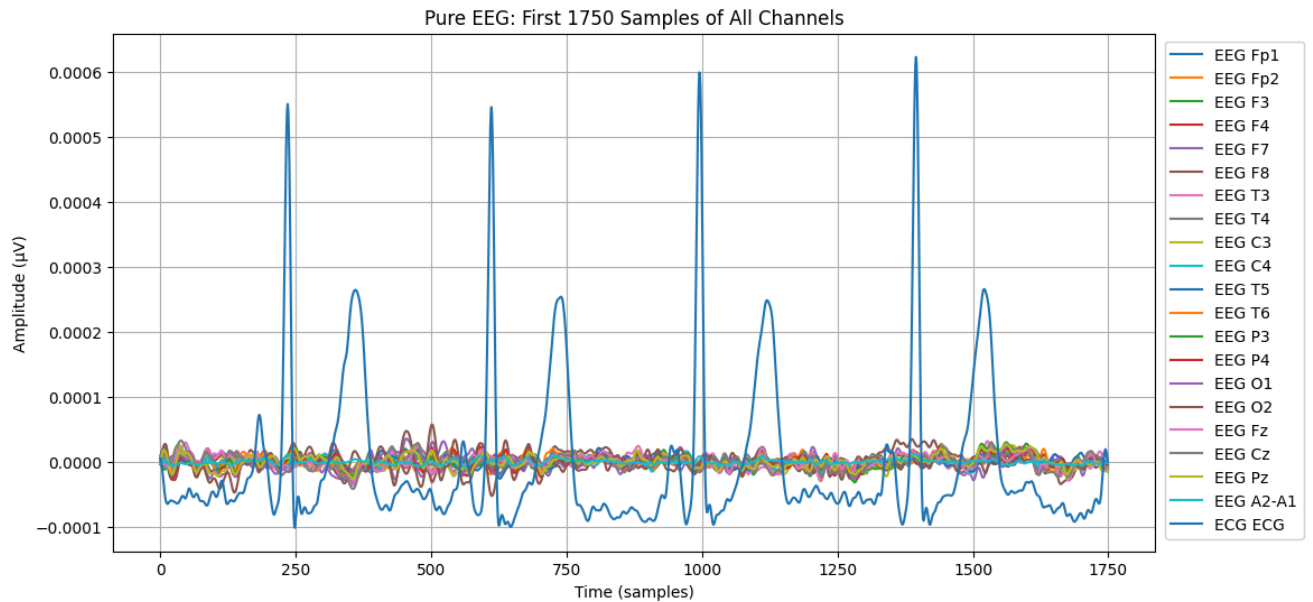
```
Extracting EDF parameters from Subject00_1.edf...
Setting channel info structure...
Creating raw.info structure...
Reading 0 ... 90999 = 0.000 ... 181.998 secs...
```

```
import matplotlib.pyplot as plt

eeg_sample = data[:, :1750] # all channels, first 1750 samples

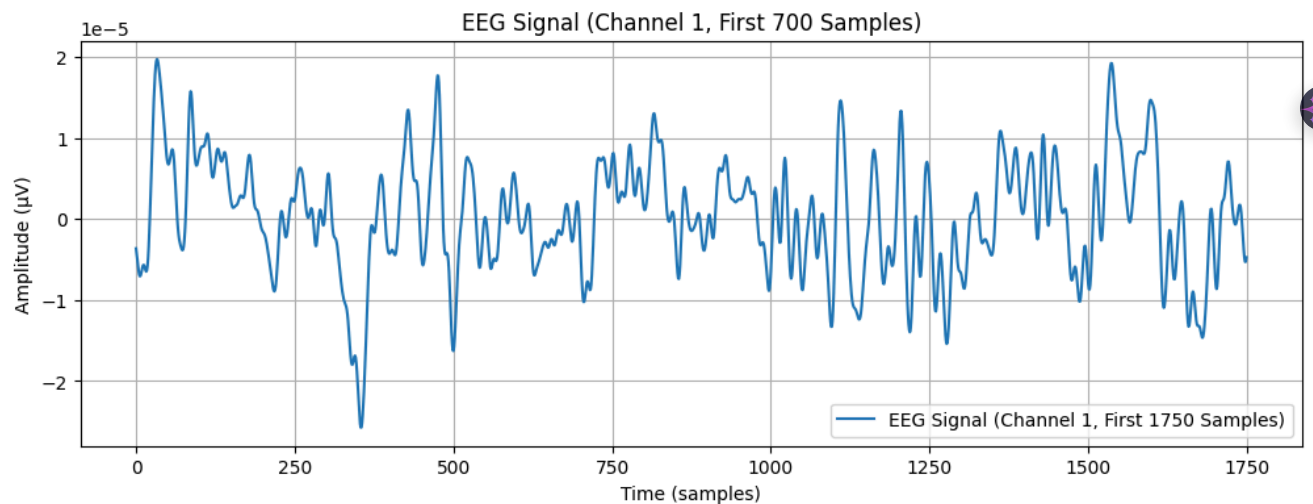
plt.figure(figsize=(12, 6))
for ch in range(eeg_sample.shape[0]):
    plt.plot(eeg_sample[ch, :], label=raw.ch_names[ch])

plt.xlabel('Time (samples)')
plt.ylabel('Amplitude (µV)')
plt.title('Pure EEG: First 1750 Samples of All Channels')
plt.legend(loc='upper right', bbox_to_anchor=(1.15, 1))
plt.grid(True)
plt.show()
```

```
import matplotlib.pyplot as plt
```

```
Pure_FP1 = data[0, :1750]
# Plot
plt.figure(figsize=(12, 4))
plt.plot(Pure_FP1, label='EEG Signal (Channel 1, First 1750 Samples)')
plt.xlabel('Time (samples)')
plt.ylabel('Amplitude (µV)')
plt.title('EEG Signal (Channel 1, First 700 Samples)')
plt.legend()
plt.grid(True)
plt.show()
```

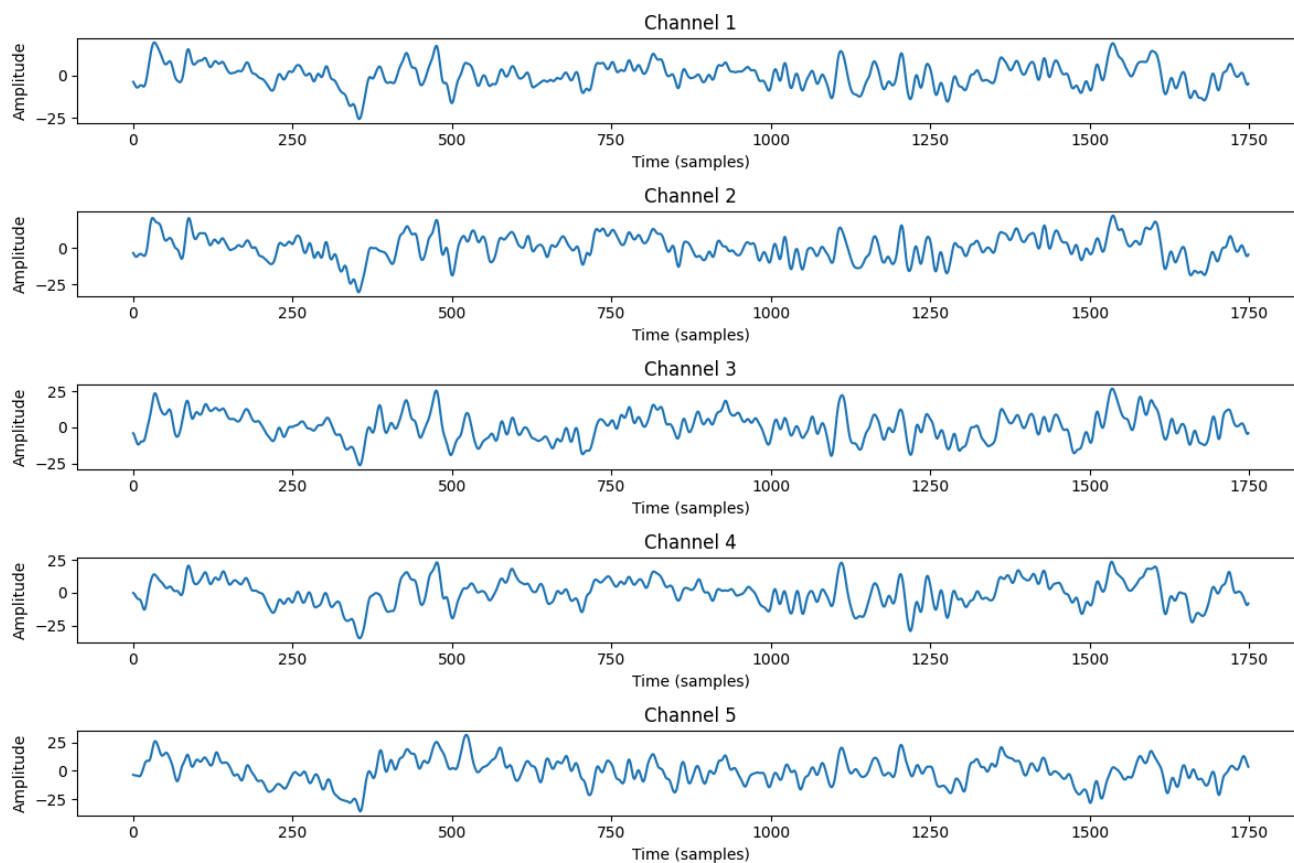


```
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(12, 8))
```

```
for i in range(5):
    plt.subplot(5, 1, i+1)
    plt.plot(data[i, :1750] * 1e6)
    plt.title(f'Channel {i+1}')
    plt.xlabel('Time (samples)')
    plt.ylabel('Amplitude')
```

```
plt.tight_layout()
plt.show()
```



```
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Prepare data from the MNE array (Channel 0, first 3.5 seconds)
# N should be 1750 for 3.5s at 500 Hz
Pure_FP1 = data[0, :1750] * 1e6
N = len(Pure_FP1)
sampling_rate = 500

frequencies = np.fft.fftfreq(N, d=1/sampling_rate)

# Step 2: Initialize components
real_part = np.zeros(N)
imaginary_part = np.zeros(N)

# Step 3: Implement DFT (Indentation fixed)
for k in range(N):
    real_sum = 0
    imag_sum = 0
    for n in range(N):
        angle = 2 * np.pi * k * n / N
        real_sum += Pure_FP1[n] * np.cos(angle)
        imag_sum -= Pure_FP1[n] * np.sin(angle)

    real_part[k] = real_sum
    imaginary_part[k] = imag_sum

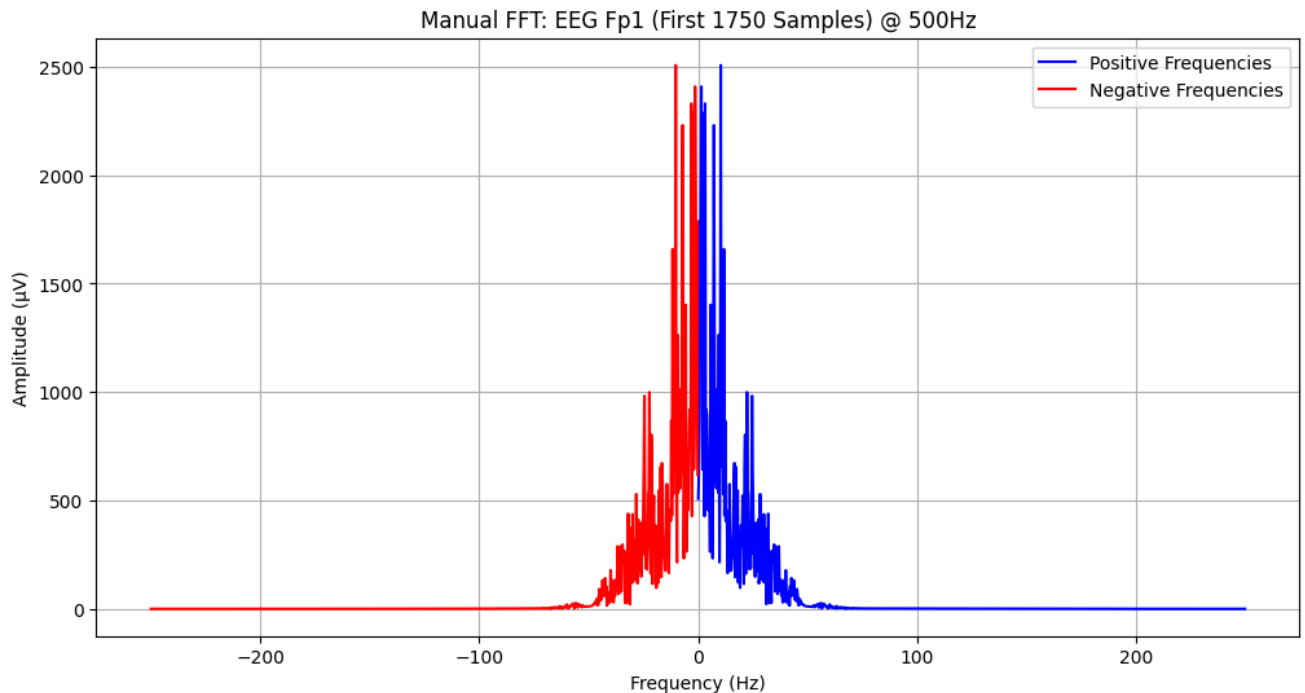
# Step 4: Calculate magnitude
amplitude = np.sqrt(real_part**2 + imaginary_part**2)

# Step 5: Plotting (Updated for 1750 samples)
```



```
plt.figure(figsize=(12, 6))
plt.plot(frequencies[:N//2], amplitude[:N//2], label="Positive Frequencies", color='blue')
plt.plot(frequencies[N//2:], amplitude[N//2:], label="Negative Frequencies", color='red')

plt.title(f'Manual FFT: {raw.ch_names[0]} (First 1750 Samples) @ 500Hz')
plt.xlabel('Frequency (Hz)')
plt.ylabel('Amplitude (μV)')
plt.grid(True)
plt.legend()
plt.show()
```



```
import numpy as np
import matplotlib.pyplot as plt

sampling_rate = 500
# Select first channel, first 3.5 seconds, and scale to μV
Pure_FP1 = data[0, :1750] * 1e6
N = len(Pure_FP1)
frequencies = np.linspace(0, sampling_rate / 2, N // 2)

real_part = np.zeros(N)
imaginary_part = np.zeros(N)

for k in range(N):
    real_sum = 0
    imag_sum = 0
    for n in range(N):
        angle = 2 * np.pi * k * n / N
        real_sum += Pure_FP1[n] * np.cos(angle)
        imag_sum -= Pure_FP1[n] * np.sin(angle)

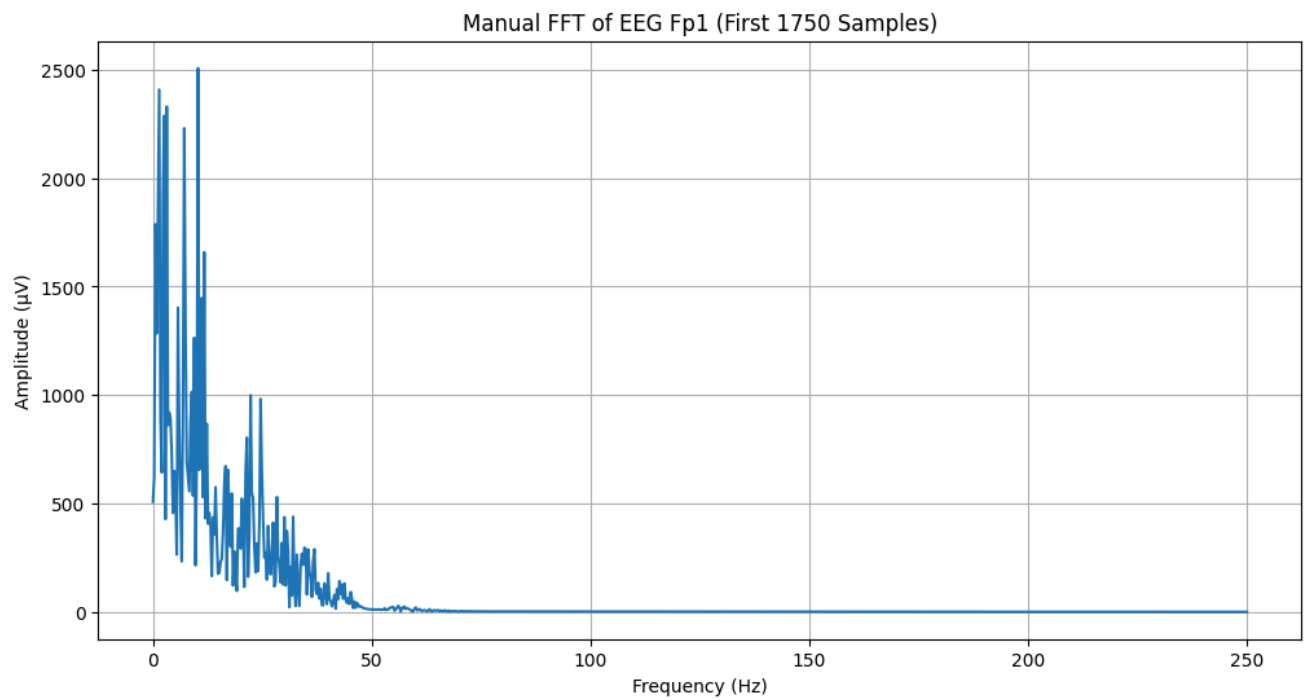
    real_part[k] = real_sum
    imaginary_part[k] = imag_sum

amplitude = np.sqrt(real_part**2 + imaginary_part**2)

plt.figure(figsize=(12, 6))
plt.plot(frequencies, amplitude[:N//2])

plt.title(f'Manual FFT of {raw.ch_names[0]} (First 1750 Samples)')
plt.xlabel('Frequency (Hz)')
plt.ylabel('Amplitude (μV)')
plt.grid(True)
plt.show()
```





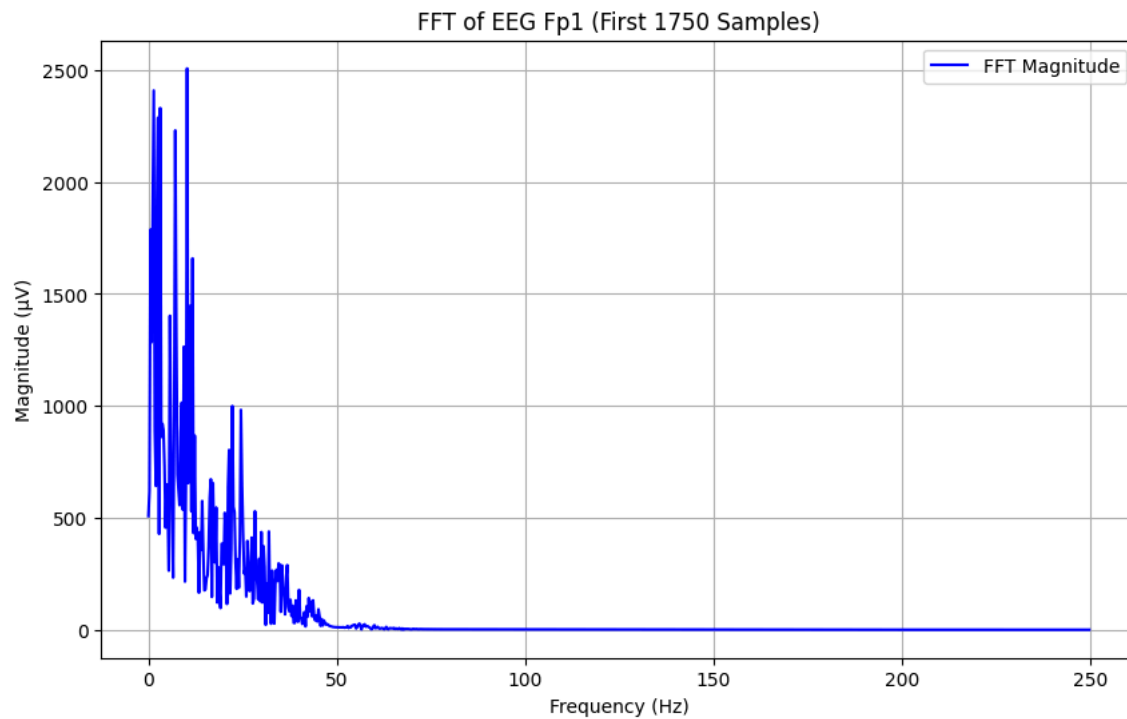
```
import numpy as np
import matplotlib.pyplot as plt

# 1750 samples = 3.5 seconds at 500 Hz
segment = Pure_FP1[:1750]
N = len(segment)

# FFT
fft_values = np.fft.fft(segment)
fft_magnitude = np.abs(fft_values)[:N//2]
sampling_rate = 500
freqs = np.fft.fftfreq(N, d=1/sampling_rate)[:N//2]

# Plot
plt.figure(figsize=(10,6))
plt.plot(freqs, fft_magnitude, color='blue', label='FFT Magnitude')
plt.title(f'FFT of {raw.ch_names[0]} (First 1750 Samples)')
plt.xlabel('Frequency (Hz)')
plt.ylabel('Magnitude (µV)')
plt.grid(True)
plt.legend()
plt.show()
```





```
raw = mne.io.read_raw_edf('Subject00_2.edf', preload=True)
cont_data = raw.get_data() * 1e6
sampling_rate = 500
N = 1750
```

```
Extracting EDF parameters from Subject00_2.edf...
Setting channel info structure...
Creating raw.info structure...
Reading 0 ... 30999 = 0.000 ... 61.998 secs...
```

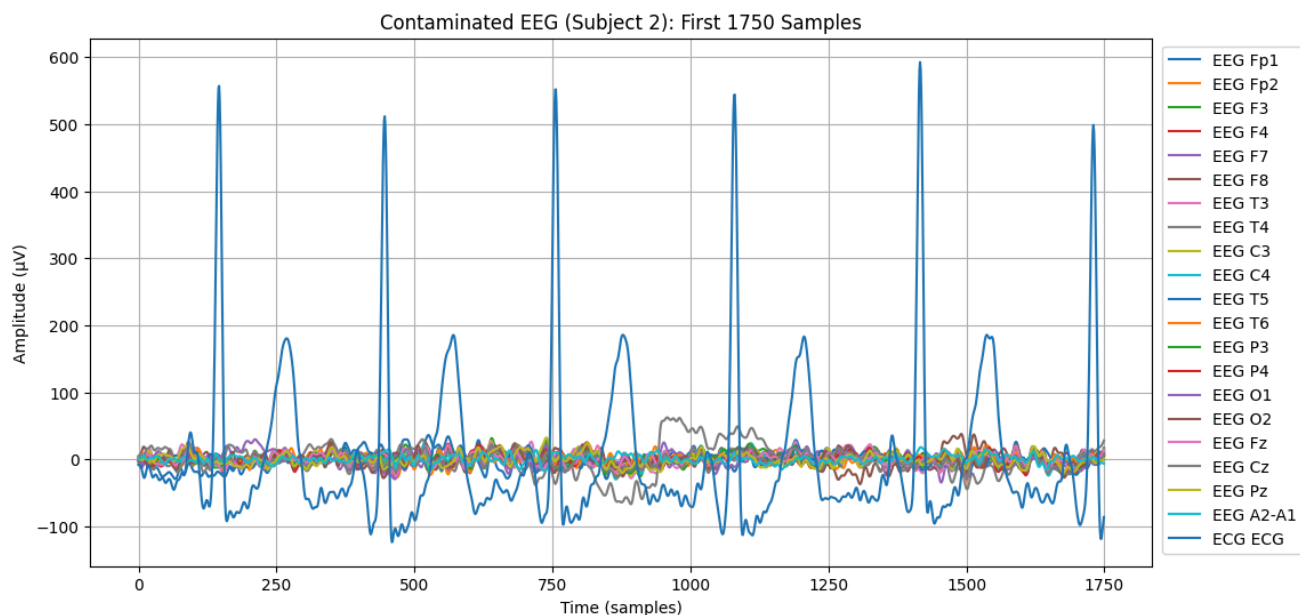
```
import matplotlib.pyplot as plt

eeg_sample1 = cont_data[:, :N]

plt.figure(figsize=(12, 6))
for ch in range(eeg_sample1.shape[0]):
    plt.plot(eeg_sample1[ch, :], label=raw.ch_names[ch])

plt.xlabel('Time (samples)')
plt.ylabel('Amplitude (μV)')
plt.title('Contaminated EEG (Subject 2): First 1750 Samples')
plt.legend(loc='upper right', bbox_to_anchor=(1.15, 1))
plt.grid(True)
plt.show()
```

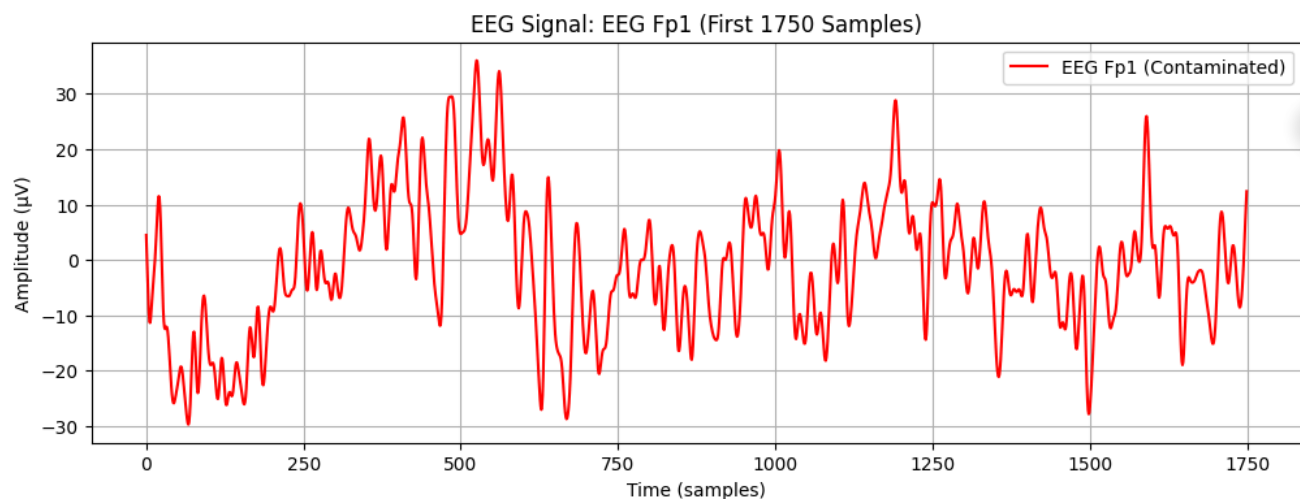




```
import matplotlib.pyplot as plt
```

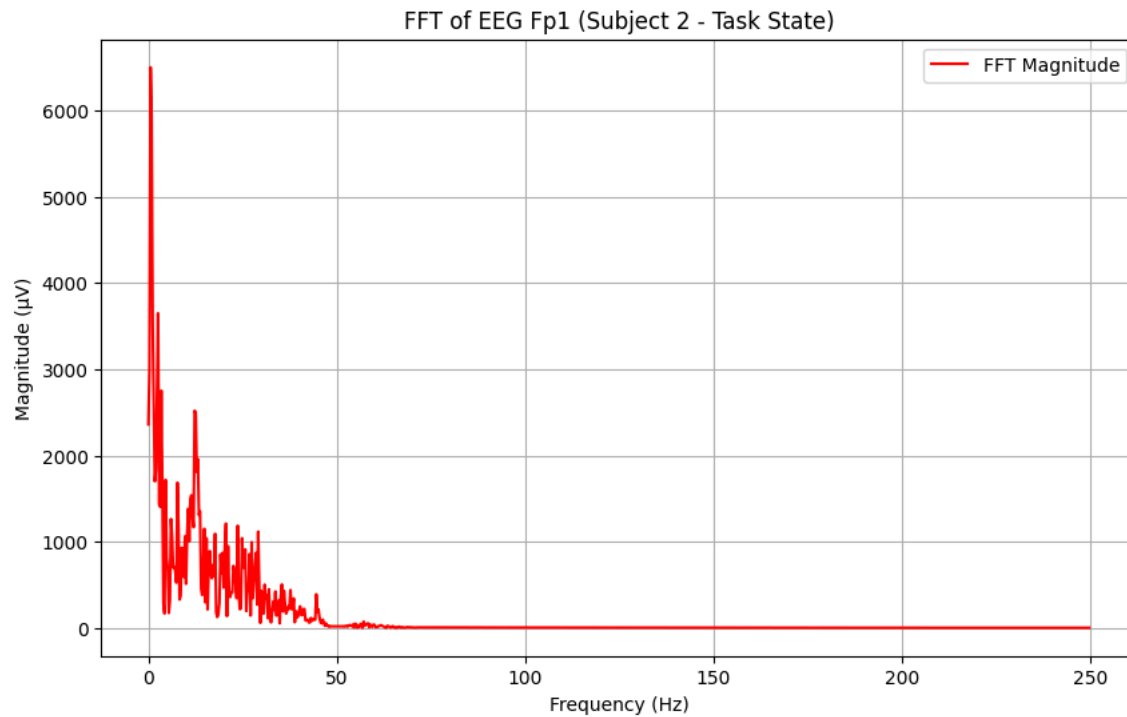
```
Cont_FP1 = cont_data[0, :N]
```

```
plt.figure(figsize=(12, 4))
plt.plot(Cont_FP1, label=f'{raw.ch_names[0]} (Contaminated)', color='red')
plt.xlabel('Time (samples)')
plt.ylabel('Amplitude ( $\mu\text{V}$ )')
plt.title(f'EEG Signal: {raw.ch_names[0]} (First 1750 Samples)')
plt.legend()
plt.grid(True)
plt.show()
```



```
fft_values = np.fft.fft(Cont_FP1)
fft_magnitude1 = np.abs(fft_values)[:N//2]
freqs = np.fft.fftfreq(N, d=1/sampling_rate)[:N//2]

plt.figure(figsize=(10, 6))
plt.plot(freqs, fft_magnitude1, color='red', label='FFT Magnitude')
plt.title(f'FFT of {raw.ch_names[0]} (Subject 2 - Task State)')
plt.xlabel('Frequency (Hz)')
plt.ylabel('Magnitude ( $\mu$ V)')
plt.grid(True)
plt.legend()
plt.show()
```



```
import matplotlib.pyplot as plt

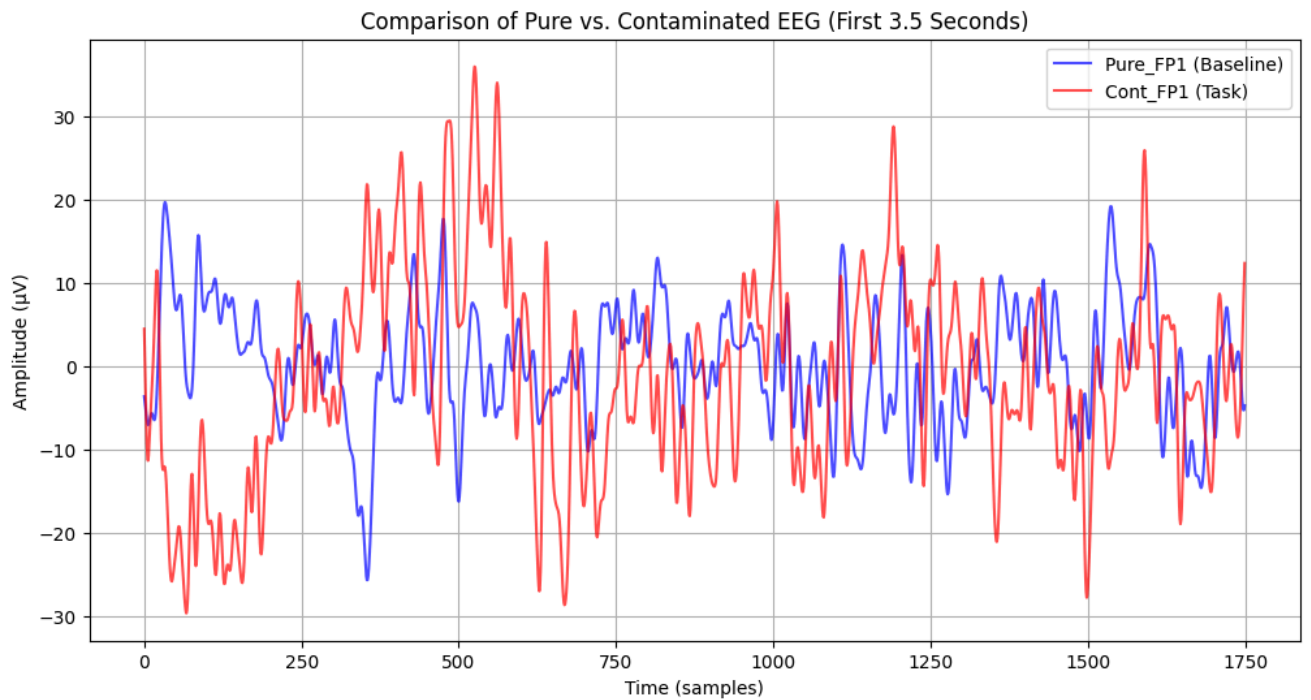
plt.figure(figsize=(12, 6))

# Use 1750 samples to maintain a 3.5-second window at 500 Hz
# Ensure Pure_FP1 and Cont_FP1 were scaled (e.g., data * 1e6)
plt.plot(Pure_FP1[:1750], label='Pure_FP1 (Baseline)', color='blue', alpha=0.7)
plt.plot(Cont_FP1[:1750], label='Cont_FP1 (Task)', color='red', alpha=0.7)

plt.ylabel('Amplitude ( $\mu$ V)')
plt.xlabel('Time (samples)')
plt.title('Comparison of Pure vs. Contaminated EEG (First 3.5 Seconds)')

# The legend labels are automatically pulled from the 'label' argument in plt.plot
plt.legend(loc="upper right")
plt.grid(True)
plt.show()
```

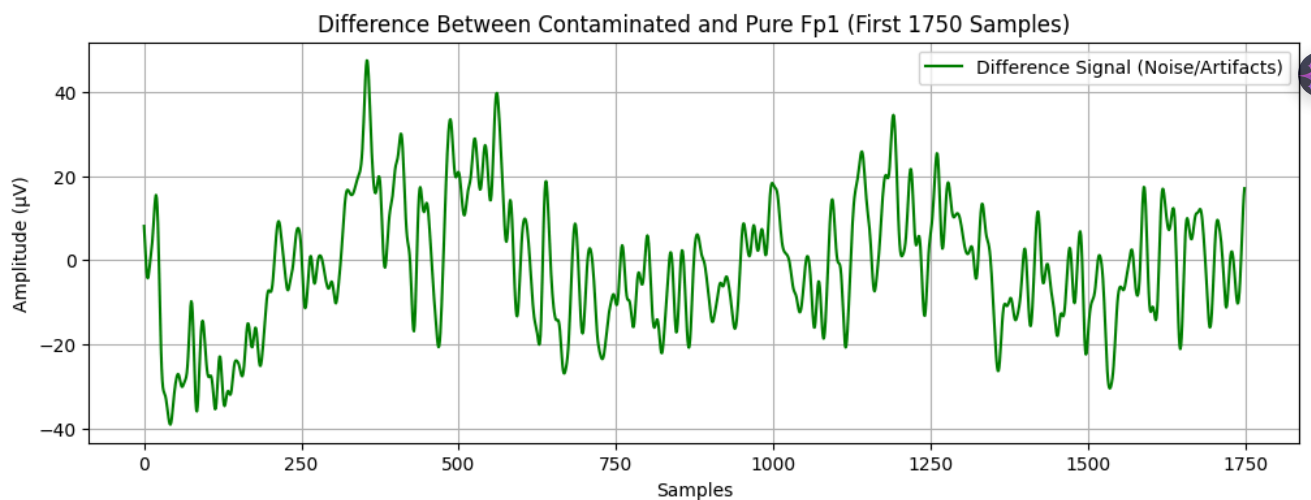




```
import matplotlib.pyplot as plt
```

```
# Update to 1750 samples to match the 3.5-second window at 500 Hz
difference_signal = Cont_FP1[:1750] - Pure_FP1[:1750]
```

```
plt.figure(figsize=(12, 4))
plt.plot(difference_signal, label="Difference Signal (Noise/Artifacts)", color='green')
plt.title("Difference Between Contaminated and Pure Fp1 (First 1750 Samples)")
plt.xlabel("Samples")
plt.ylabel("Amplitude ( $\mu\text{V}$ )")
plt.grid(True)
plt.legend()
plt.show()
```



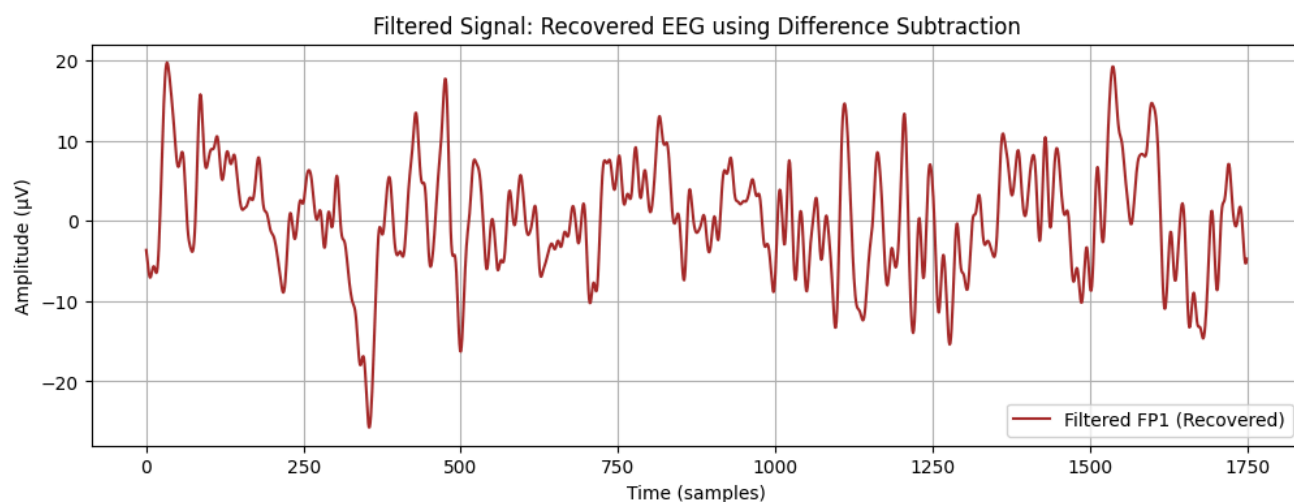
```
import matplotlib.pyplot as plt
```

```
# 1. Ensure the slice matches the 1750 samples (3.5s at 500 Hz)
# 2. Subtract the difference_signal calculated in the previous step
Filt_FP1 = Cont_FP1[:1750] - difference_signal
```

```
plt.figure(figsize=(12, 4))
plt.plot(Filt_FP1, label="Filtered FP1 (Recovered)", color='brown')

plt.title("Filtered Signal: Recovered EEG using Difference Subtraction")
```

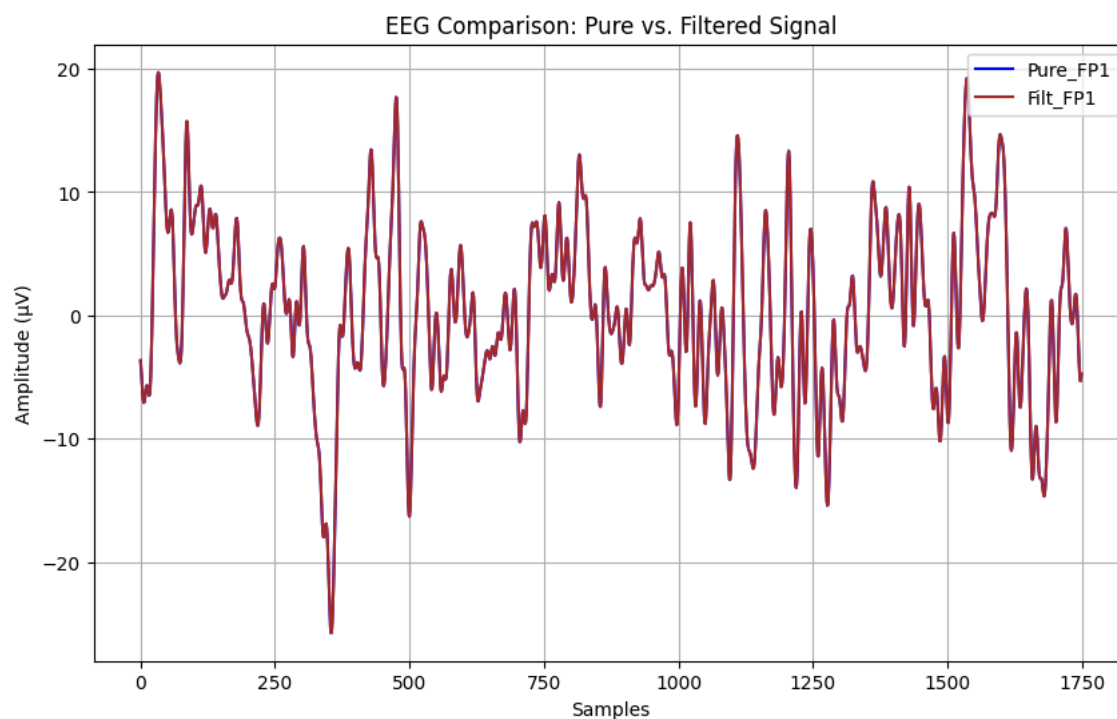
```
plt.xlabel("Time (samples)")
plt.ylabel("Amplitude (μV)")
plt.grid(True)
plt.legend()
plt.show()
```



```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))
plt.plot(Pure_FP1[:1750], label='Pure_FP1', color='blue')
plt.plot(Filt_FP1[:1750], label='Filt_FP1', color='brown')

plt.ylabel('Amplitude (μV)')
plt.xlabel('Samples')
plt.title('EEG Comparison: Pure vs. Filtered Signal')
plt.legend(loc="upper right")
plt.grid(True)
plt.show()
```



✖ remove 50 Hz power line interference

```

from scipy.signal import iirnotch, filtfilt
import matplotlib.pyplot as plt
import numpy as np

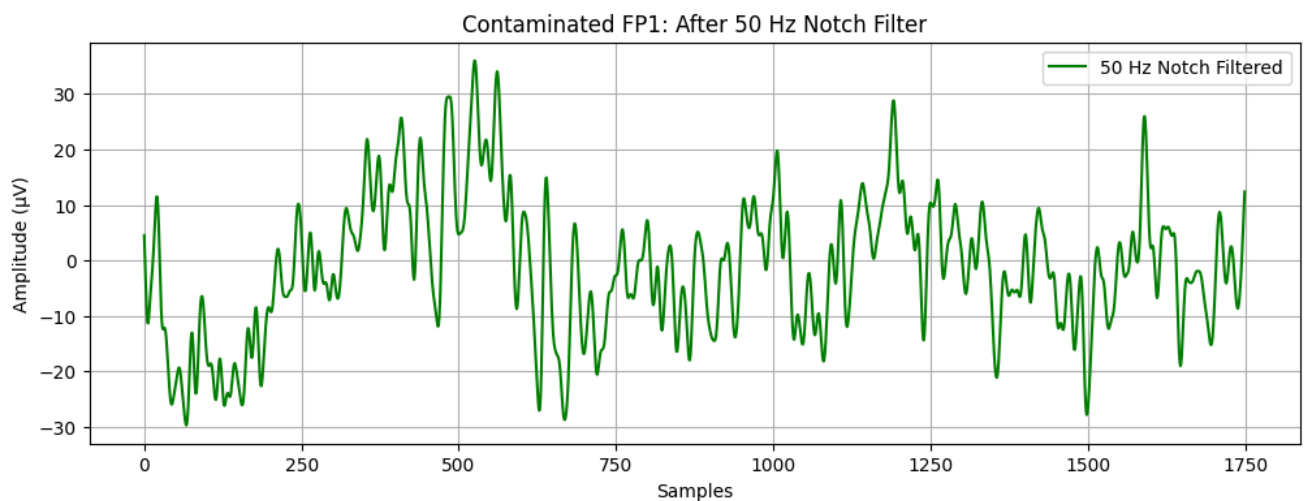
fs = 500
f0 = 50.0 # Frequency to be removed (Power line interference)
Q = 30.0 # Quality factor

# Notch filter design
b, a = iirnotch(f0, Q, fs)

# Apply the filter (assuming Cont_FP1 is scaled to  $\mu\text{V}$ )
Cont_FP1_notched = filtfilt(b, a, Cont_FP1)

plt.figure(figsize=(12, 4))
plt.plot(Cont_FP1_notched[:1750], label='50 Hz Notch Filtered', color='green')
plt.xlabel('Samples')
plt.ylabel('Amplitude ( $\mu\text{V}$ )')
plt.title('Contaminated FP1: After 50 Hz Notch Filter')
plt.legend()
plt.grid(True)
plt.show()

```



```

N = len(Cont_FP1_notched)

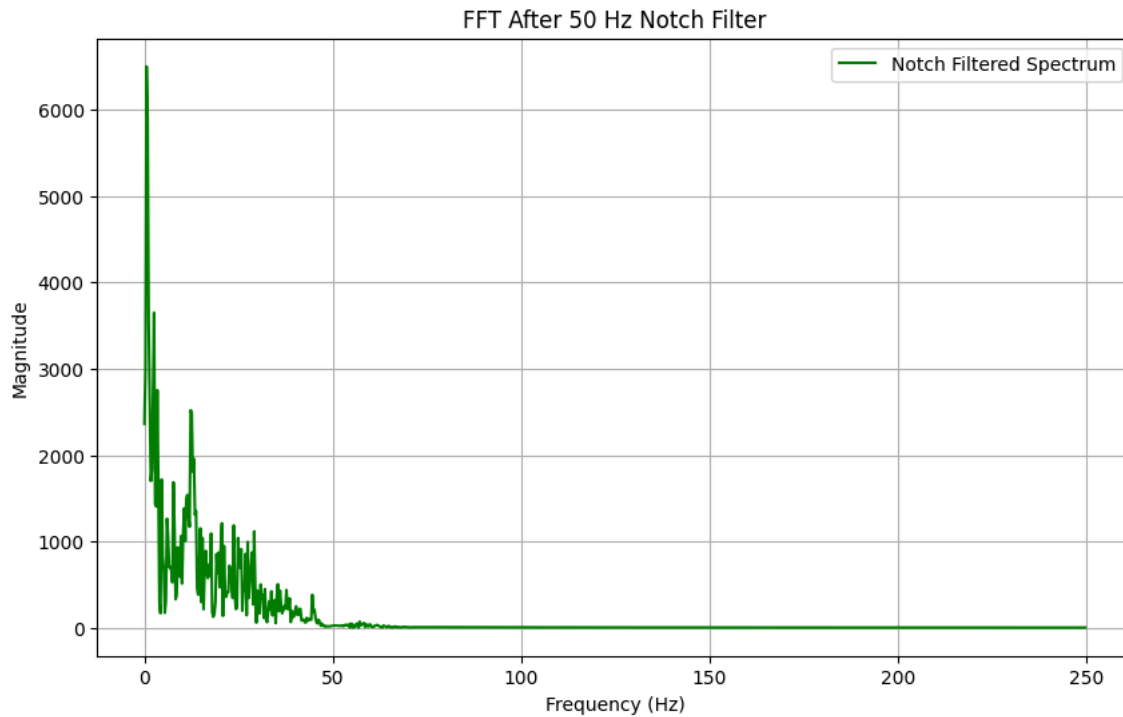
freqs = np.fft.fftfreq(N, d=1/fs)
fft_notch = np.abs(np.fft.fft(Cont_FP1_notched))[:N//2]

plt.figure(figsize=(10, 6))

plt.plot(freqs[:N//2], fft_notch, label='Notch Filtered Spectrum', color='green')
plt.xlabel('Frequency (Hz)')
plt.ylabel('Magnitude')
plt.title('FFT After 50 Hz Notch Filter')
plt.grid(True)
plt.legend()
plt.show()

```





For OCCular artifacts

```
import numpy as np
import matplotlib.pyplot as plt

# 1. Update parameters for 500 Hz and 3.5 seconds of data
sampling_rate = 500
N = 1750

# 2. Extract segments and ensure they are scaled (assuming  $\mu V$ )
Pure_segment = Pure_FP1[:N]
segment = Cont_FP1[:N]

# 3. Compute FFTs
# Note: N must be defined before slicing [:N//2]
fft_cont = np.fft.fft(segment)
fft_cont_mag = np.abs(fft_cont)[:N//2]

fft_pure = np.fft.fft(Pure_segment)
fft_pure_mag = np.abs(fft_pure)[:N//2]

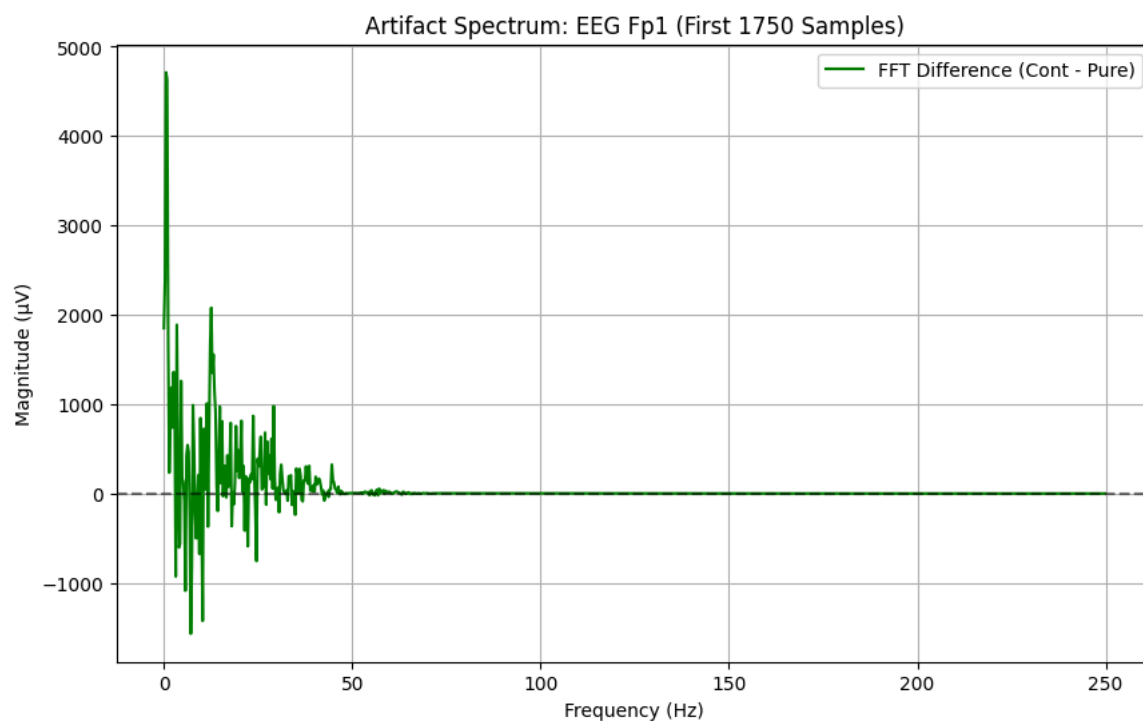
# 4. Artifact spectrum calculation
# This isolates frequencies where the contaminated signal has higher energy
fft_diff = fft_cont_mag - fft_pure_mag

# 5. Frequency axis generation
freqs = np.fft.fftfreq(N, d=1/sampling_rate)[:N//2]

# Plotting
plt.figure(figsize=(10,6))
plt.plot(freqs, fft_diff, color='green', label='FFT Difference (Cont - Pure)')

plt.xlabel('Frequency (Hz)')
plt.ylabel('Magnitude ( $\mu V$ )')
plt.title(f'Artifact Spectrum: {raw.ch_names[0]} (First 1750 Samples)')
plt.grid(True)
plt.axhline(0, color='black', linestyle='--', alpha=0.5) # Zero baseline
plt.legend()
plt.show()
```





High pass Butterworth Filter

```
from scipy.signal import butter, filtfilt
import matplotlib.pyplot as plt
import numpy as np

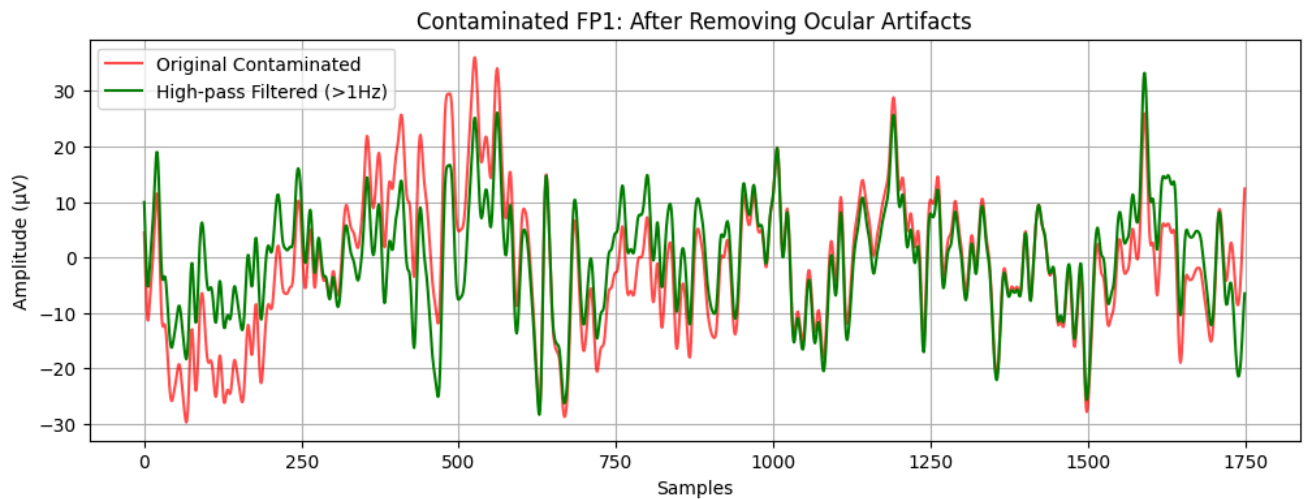
# 1. Update sampling frequency for PhysioNet
fs = 500
cutoff = 1.0
order = 4

# High-pass filter design
b, a = butter(order, cutoff/(fs/2), btype='highpass')

# Apply filter (assuming Cont_FP1 is scaled to μV)
Cont_FP1_hp = filtfilt(b, a, Cont_FP1)

plt.figure(figsize=(12,4))
plt.plot(Cont_FP1[:1750], label='Original Contaminated', color='red', alpha=0.7)
plt.plot(Cont_FP1_hp[:1750], label='High-pass Filtered (>1Hz)', color='green')
plt.xlabel('Samples')
plt.ylabel('Amplitude (μV)')
plt.title('Contaminated FP1: After Removing Ocular Artifacts')
plt.legend()
plt.grid(True)
plt.show()
```

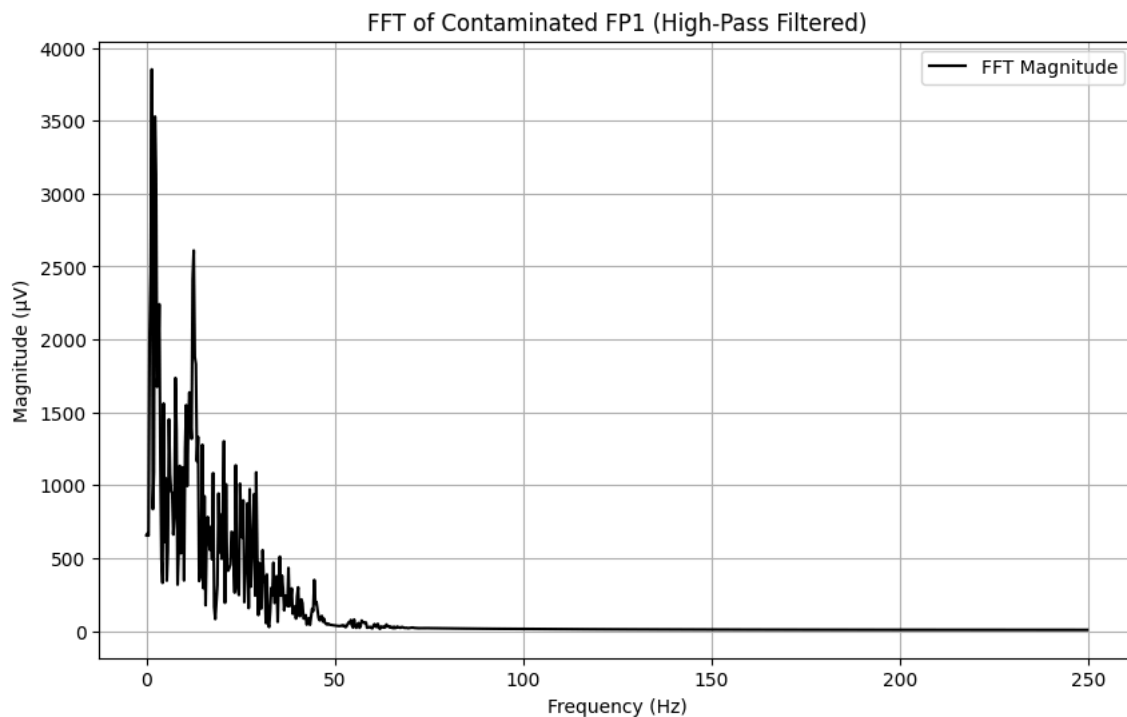




```
segment = Cont_FP1_hp[:1750]
N = len(segment)

fft_values = np.fft.fft(segment)
fft_magnitude1 = np.abs(fft_values)[:N//2]
sampling_rate = 500
freqs = np.fft.fftfreq(N, d=1/sampling_rate)[:N//2]

plt.figure(figsize=(10,6))
plt.plot(freqs, fft_magnitude1, color='black', label='FFT Magnitude')
plt.title('FFT of Contaminated FP1 (High-Pass Filtered)')
plt.xlabel('Frequency (Hz)')
plt.ylabel('Magnitude (µV)')
plt.grid(True)
plt.legend()
plt.show()
```



```
N_comp = 1750
fft_filtered = np.abs(np.fft.fft(Cont_FP1_hp[:N_comp]))[:N_comp//2]
fft_orig = np.abs(np.fft.fft(Cont_FP1[:N_comp]))[:N_comp//2]
freqs_comp = np.fft.fftfreq(N_comp, d=1/fs)[:N_comp//2]

plt.figure(figsize=(10,6))
plt.plot(freqs_comp, fft_orig, color='red', alpha=0.7, label='Original Contaminated')
```

```
plt.plot(freqs_comp, fft_filtered, color='green', label='High-pass Filtered (>1Hz)')
plt.xlabel('Frequency (Hz)')
plt.ylabel('Magnitude ( $\mu$ V)')
plt.title('FFT Comparison: Impact of High-pass Filtering')
plt.xlim(0, 10)
plt.grid(True)
plt.legend()
plt.show()
```

